

Complexity Theory: P, NP, Reductions & NP-Completeness

1 Problems: Optimization vs. Decision

Decision Problem

A decision problem returns only YES or NO. Complexity classes such as **P**, **NP**, and **NP-complete** are defined for decision problems.

Optimization version	Decision version
Find a minimum vertex cover.	Is there a vertex cover of size at most k ?
Find a maximum independent set.	Is there an independent set of size at least k ?
Find the largest clique.	Is there a clique of size at least k ?
Find the minimum TSP tour.	Is there a TSP tour of cost at most k ?

Threshold Rule

Minimum-type problems usually become “at most k ”; maximum-type problems usually become “at least k ”.

2 Polynomial Time and Class P

Polynomial Time

An algorithm is polynomial-time if its running time is bounded by

$$\mathcal{O}(n^c)$$

for some constant c , where n is the **encoded input length**.

Polynomial	Not polynomial
$\mathcal{O}(n)$, $\mathcal{O}(n \log n)$, $\mathcal{O}(n^2)$, $\mathcal{O}(n^5)$	$\mathcal{O}(2^n)$, $\mathcal{O}(n!)$

Why Polynomial Time Is the Standard Boundary

- It is stable across reasonable computation models under polynomial simulation.
- Polynomial bounds remain polynomial under composition, addition, and multiplication.
- It gives a robust theoretical notion of efficient solvability.

$$\mathcal{O}(n^a) + \mathcal{O}(n^b) = \mathcal{O}(n^{\max(a,b)}), \quad \mathcal{O}(n^a)\mathcal{O}(n^b) = \mathcal{O}(n^{a+b}).$$

Class P

$$\mathbf{P} = \{L \subseteq \{0,1\}^* : L \text{ is decidable in polynomial time}\}$$

using a deterministic algorithm/Turing machine.

Typical examples from the source notes include graph connectivity, shortest-path decision, minimum-spanning-tree decision, sorting/searching tasks, and primality testing.

3 Class NP: Certificates and Verification

Verifier Definition of NP

A language L is in **NP** if there is a polynomial-time verifier V and a polynomial p such that

$$x \in L \iff \exists y, |y| \leq p(|x|) \text{ and } V(x, y) = \text{YES.}$$

The string y is a **certificate** (or witness).

Solve vs. Verify

P: find the answer in polynomial time. **NP**: verify a proposed YES solution in polynomial time.

Problem	Certificate	Verification
SAT	truth assignment	substitute values and check the formula
CLIQUE	set of k vertices	check every pair has an edge
Vertex Cover	set of vertices	check size bound and cover every edge
Hamiltonian Cycle	ordering of vertices	check every vertex once, consecutive edges, and return edge

Why $P \subseteq NP$

If $L \in P$, use its polynomial-time decider as a verifier and ignore the certificate. Therefore

$$P \subseteq NP.$$

Name of NP

NP means **Nondeterministic Polynomial Time**, not “non-polynomial.” Membership in NP does not mean we know a polynomial-time solving algorithm.

4 P vs. NP, Complements, and co-NP

Open Question

$$P \stackrel{?}{=} NP$$

It is unknown whether every efficiently verifiable decision problem is also efficiently solvable.

Complement and co-NP

For a language L ,

$$\bar{L} = \{x \in \{0,1\}^* : x \notin L\}, \quad \text{coNP} = \{L : \bar{L} \in \text{NP}\}.$$

Known Relationships

$$P \subseteq NP, \quad P \subseteq \text{coNP}, \quad P \subseteq NP \cap \text{coNP}.$$

P is closed under complement: swap accept and reject in a polynomial-time decider.

It is also unknown whether

$$NP = \text{coNP}.$$

5 Polynomial-Time Reductions

Polynomial-Time Many-One Reduction

For languages $L_1, L_2 \subseteq \{0,1\}^*$,

$$L_1 \leq_p L_2$$

means there exists a polynomial-time computable function f such that

$$x \in L_1 \iff f(x) \in L_2.$$



How to Read $L_1 \leq_p L_2$

- L_1 can be translated to L_2 efficiently.
- A solver for L_2 gives a solver for L_1 .

- Therefore L_2 is at least as hard as L_1 .

Lemma 34.3

If

$$L_1 \leq_p L_2 \quad \text{and} \quad L_2 \in \mathbf{P},$$

then

$$L_1 \in \mathbf{P}.$$

Algorithm: compute $f(x)$ in polynomial time, solve $f(x)$ using the polynomial-time decider for L_2 , and return the same answer. Polynomial-time steps composed together remain polynomial.

Reduction Direction

To prove a target problem B is hard, reduce a known hard problem **to** B :

$$A \leq_p B.$$

Showing $B \leq_p A$ only shows that B is no harder than A .

6 NP-Hard and NP-Complete

NP-Hard

A problem H is **NP-hard** if

$$\forall L \in \mathbf{NP}, \quad L \leq_p H.$$

It does not have to be a decision problem and does not have to belong to **NP**.

NP-Complete

A decision problem C is **NP-complete** exactly when

$$C \in \mathbf{NP} \quad \text{and} \quad C \text{ is NP-hard.}$$

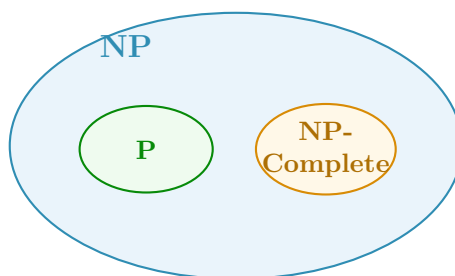
Equivalently, every language in **NP** polynomial-time reduces to C .

Class	Requirement	Meaning
P	solvable in polynomial time	efficiently solvable
NP	polynomial-size YES certificate verifiable in polynomial time	efficiently verifiable
NP-hard	every NP problem reduces to it	at least as hard as all of NP
NP-complete	in NP and NP-hard	hardest problems inside NP

Decision vs. Optimization Example

TSP decision: “Is there a tour of cost at most k ?” – **NP-complete** in the source notes.

TSP optimization: “Find the minimum-cost tour.” – **NP-hard** in the source notes.



NP-complete problems are inside NP; whether $P = NP$ is unknown.

7 Why NP-Completeness Matters

Theorem 34.4

If any **NP**-complete problem C is in **P**, then

$$\boxed{\mathbf{P} = \mathbf{NP}.}$$

Proof: for any $L \in \mathbf{NP}$, NP-completeness gives $L \leq_p C$. If $C \in \mathbf{P}$, Lemma 34.3 gives $L \in \mathbf{P}$. Hence $\mathbf{NP} \subseteq \mathbf{P}$. Together with $\mathbf{P} \subseteq \mathbf{NP}$, we get equality.

This is why one polynomial-time algorithm for one NP-complete problem would give polynomial-time algorithms for all problems in NP.

8 How to Prove a New Problem Is NP-Complete

Standard Proof Pattern

For a target decision problem B :

1. Show $B \in \mathbf{NP}$ by giving a polynomial-size certificate and polynomial-time verifier.
2. Choose a known NP-complete problem A .
3. Construct a polynomial-time mapping f proving

$$A \leq_p B.$$

4. Prove both directions:

$$x \in A \iff f(x) \in B.$$

5. Conclude B is NP-hard; with $B \in \mathbf{NP}$, conclude B is NP-complete.

Compact Proof Logic

$$\boxed{B \in \mathbf{NP}} + \boxed{A \leq_p B \text{ for known NPC } A} \implies \boxed{B \text{ is NP-complete.}}$$

9 Unified Reference Table

Term	Precise idea
Polynomial time	$\mathcal{O}(n^c)$ for constant c , measured in encoded input length.
Abstract problem	Mathematical relation between instances and solutions.
Encoding	Map from abstract instances to binary strings.
Concrete problem	Problem whose inputs are binary strings.
P	Decision problems solvable in polynomial time.
Certificate	Polynomial-size evidence for a YES instance.
Verifier	Polynomial-time algorithm checking (x, y) .
NP	Decision problems with polynomial-size YES certificates verifiable in polynomial time.
coNP	Languages whose complements belong to NP .
$L_1 \leq_p L_2$	Poly-time translation preserving YES/NO answers; L_2 is at least as hard.
NP-hard	Every problem in NP reduces to it.
NP-complete	In NP and NP-hard.

Core Relationships

$$\boxed{\mathbf{P} \subseteq \mathbf{NP}}, \quad \boxed{\mathbf{P} \subseteq \mathbf{coNP}}, \quad \boxed{\mathbf{NP-complete} = \mathbf{NP} \cap \mathbf{NP-hard} \text{ (for decision problems)}}$$

$$\boxed{L_1 \leq_p L_2 \ \& \ L_2 \in \mathbf{P} \Rightarrow L_1 \in \mathbf{P}}, \quad \boxed{\text{one NPC problem in } \mathbf{P} \Rightarrow \mathbf{P} = \mathbf{NP}.}$$